

Intro a Desarrollo Web con HTML

Como apoyo para el tema puedes usar la siguiente referencia [HTML for Babies](#).

Plantilla básica de HTML

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>HTML</title>
  </head>
  <body>
    <!-- Content -->
  </body>
</html>
```

Introducción a HTML

El desarrollo web se basa en programación por etiquetas, estas vienen de la forma.

```
<tag></tag>
```

Esta sintaxis fue creada por el creador de la WWW Tim Berners Lee, y se ha vuelto el modelo estándar de desarrollo web que los navegadores hoy en día utilizan para todo lo que hoy en día se encuentra disponible en Internet.

Si bien los navegadores web utilizan el HTML como estándar, no es el único protocolo de comunicación para obtener información de un servidor.

¡Recuerda! En desarrollo web la unidad mínima de trabajo es una arquitectura Cliente-Servidor, desde la cual el cliente funciona como nuestro navegador y desde aquí empezaremos a hacer peticiones a un servidor del cual podemos obtener archivos estáticos (html,css,js,jpg,png,etc.) o resultados procesados por la entidad de nuestro servidor.

Es muy importante que desde ahora tomes en cuenta en donde se encuentra la información y en que momento para que puedas resolver errores, es decir si estoy viendo un HTML ¿quién lo está cargando?, ¿el servidor me ha devuelto una respuesta?, preguntas como las anteriores nos ayudan a ubicar y entre mas conceptos añadimos más fácil se vuelve añadir más complejidad.

Estándares adicionales de transmisión de información HTML, XML, JSON

Para entender este punto debemos localizarnos en las capas del modelo de redes OSI e identificar el protocolo TCP/IP.

Este protocolo envía paquetes de información entre el cliente y el servidor y esto nos genera lo que conocemos como **requests**.

Los requests no son más que llamadas url que nosotros escribimos en el navegador o que mandamos llamar con cada interacción que tenemos en una página web.

Es entonces que lo que vamos a procesar en el servidor son estas urls, y todas aquellas que no se encuentren son lo que conocemos como el famoso **404 not found**, más adelante veremos estos códigos de error en profundidad.

El siguiente paso al hacer un requests es que debemos saber que nos debe regresar algo, incluso si ocurre un error, y lo ideal es que nos devuelva en un formato estándar que podamos entender.

El más básico ya lo mencionamos es HTML

```
<html>
  <head>
    ...
  </head>
  <body>
    ...
  </body>
</html>
```

Otro estándar utilizado en la industria que si bien es más viejo más no es obsoleto es XML. Este estándar tiene un cierto parecido con HTML en que utiliza un sistema de etiquetas, más la diferencia con HTML es que este se centra en modelar información general, que es definida por los ingenieros de software.

```
<cuerpo>
  <cabeza></cabeza>
  <torso></torso>
  <piernas></piernas>
</cuerpo>
```

Como puedes ver HTML tiene su sintaxis específica como si fuera un lenguaje de programación, mientras que XML puede modelar cualquier tipo de dato con la diferencia que esto no será visible en el navegador.

Por último el otro tipo de estándar más usado es conocido como JSON, este estándar es más nuevo y busca ser una evolución a XML en el sentido de reducir el duplicado con el uso de etiquetas manejando la información en una estructura de datos como la pudiéramos encontrar en otros lenguajes de programación: listas, objetos, diccionarios o tablas hash. [Ver estándar JSON](#)

```
{
  "cuerpo":{
    "cabeza":{}
    "torso":{}
    "piernas":{}
  }
}
```

Existen muchos otros estándares de manejo de información para TCP/IP que puedes encontrar por ejemplo uno muy popular últimamente es GraphQL, cada uno tiene sus reglas y veremos algunos durante el curso.

Etiquetas HTML

Como ya mencionamos HTML utiliza las etiquetas para poder funcionar, para poder utilizarlas tenemos etiquetas que abren y cierran o etiquetas que cierran en la misma.

```
<tag></tag> <!--Etiqueta que abre y cierra-->

<tag/> <!-- Etiqueta que abre y cierra al declararse -->
```

La diferencia entre cada una es que la primera admite contenido adentro de la misma, mientras que la segunda solo utiliza la etiqueta para manejar su información sin utilizar contenido interno.

Es así que la etiqueta más básica en HTML podríamos manejarla como la siguiente

```
<h1> HTML For Babies</h1>
```

Atributos HTML

Una vez que tenemos una etiqueta básica veremos que dentro de los símbolos de la etiqueta que abre es decir, dentro de <> podemos tener atributos, estos dependerán de cada etiqueta, algunos son globales y otros son específicos.

Este es un ejemplo básico:

```
<ol id="family">
  <li>Baby</li>
  <li>Daddy</li>
  <li>Mommy</li>
</ol>
```

Aquí estamos declarando el atributo id cuyo valor será el de "family".

Los atributos globales que toda propiedad de HTML puede tener son el **id** y el **class**.

- El id como su nombre indica es un identificador único, y así como en los lenguajes de programación con las variables la idea es que solo exista 1 por archivo HTML, esto ayuda para hacer referencia específica a una etiqueta cuando usamos estilos CSS o para acceder a un valor si lo llamamos desde un archivo JS.
- El class, es un atributo utilizado más por el estilo CSS, este lo veremos con más profundidad en próximos laboratorios.

Otro ejemplo de etiqueta en HTML usando el atributo id sería una tabla como la siguiente

```
<table id="bday">
  <tbody>
    <tr>
      <td>Baby</td>
      <td>1 🎂</td>
    </tr>
  </tbody>
</table>
```

Para ver algunos otro atributos podemos ver el siguiente ejemplo con la etiqueta **a**

```
<a href="https://html6.com"
  target="_blank"
  rel="nofollow"
  title="HTML6">
  HTML Editor </a>
```

Las propiedades que tenemos es el href, target, rel,title. Primero entendamos que la etiqueta a sirve para formar un hipervínculo dentro de la página, es decir es un link que nos lleva a otro lugar.

- El atributo href recibe la url de la página a donde queremos ir.
- El atributo target es opcional y en este caso abre una nueva pestaña del navegador con la indicación **_blank**.
- El atributo rel igualmente es opcional y por default una etiqueta a utiliza el nofollow, esto sirve para que en caso de los buscadores como Google cuando entran a revisar las páginas y detectan estas etiquetas crean un mapa de a que sitios lleva la página, esto ayuda a la página destino a posicionarse mejor en internet, existe todo un tema al respecto dentro de marketing y SEO (Search Engine Optimization) para que funcione correctamente.
- El atributo title ayuda a mostrar al usuario un título si pasa por encima el cursor.

Formularios HTML

Además de las páginas como un todo HTML, lo más importante a manejar son los **form** o formularios, ya que estos son los que permiten guardar información una página web.

La estructura básica de un formulario completo es la siguiente

```
<form action="/age.php" method="post">
  Age: <input value="2"
    min="0" max="99"
    id="age" step="1"
    type="number" />
```

```
<input type="submit" value="Submit" />
</form>
```

La etiqueta form recibe varios atributos importantes

```
<form action="/age.php" method="post"></form>
```



- El **action** es la url que vamos a llamar y desde donde se subirá la información que vamos a enviar del formulario.
- El **method** será el tipo de request que hará la url es decir podrá ser de tipo: GET,POST,PUT,DELETE.

Nota: Cuando llames la url, asegúrate que tu servidor pueda manejar la llamada, el error más común es hacer una llamada POST y que el servidor solo procese la url mediante un GET.

En el ejemplo de código la url es **/age.php**, esto funciona para servidores desarrollados en php, pero otro tipo de urls pueden ser de formato sin extensión como **/age** solamente.

Dentro del formulario observa que tenemos etiquetas de tipo **input**, todas estas etiquetas tienen gran variación de parámetros ya que nos permiten hacer campos de texto, listas, botones, entre otros. Investiga las posibilidades que puedes realizar con los input y podrás guardar cualquier tipo de información incluso archivos en tu servidor y tu base de datos.